

```
// ===== WEATHER GATE v1.5 =====  
// С поддержкой FSK для WS1080 на 915 MHz
```

```
#define MQTT_MAX_PACKET_SIZE 1024
```

```
#include <ETH.h>  
#include <WiFi.h>  
#include <Wire.h>  
#include <RTCLib.h>  
#include <SPI.h>  
#include <LittleFS.h>  
#include <ArduinoJson.h>  
#include <WebServer.h>  
#include <PubSubClient.h>  
#include <Preferences.h>  
#include <HTTPClient.h>  
#include <HTTPUpdate.h>  
#include <time.h>  
#include <ELECHOUSE_CC1101_SRC_DRV.h>  
#include <rtl_433_ESP.h>
```

```
// ===== КОНФИГУРАЦИЯ =====  
SET_LOOP_TASK_STACK_SIZE(20 * 1024);  
#define CURRENT_VERSION "1.5"
```

```
// ===== ЧАСОВОЙ ПОЯС =====  
#define TIMEZONE_OFFSET 7
```

```
// ===== ПИНЫ WT32-ETH01 =====  
#define I2C_SDA 32  
#define I2C_SCL 33  
#define LED_RF_RX 25  
#define LED_MQTT_TX 26  
#define PIN_BAT_ADC 36  
#define PIN_POWER_SENSE 39
```

```
// Пины CC1101 (SPI)  
#define CC1101_CS 2  
#define CC1101_GDO0 4  
#define CC1101_SCK 12  
#define CC1101_MISO 15  
#define CC1101_MOSI 14
```

```
// ===== НАСТРОЙКИ =====
```

```

#define POINTS_HISTORY 288
#define HISTORY_INTERVAL 300000
#define SENSOR_INTERVAL 60000
#define MQTT_PUBLISH_INTERVAL 60000
#define MQTT_RECONNECT_INTERVAL 10000
#define SMART_SAVE_NORMAL 300000
#define SMART_SAVE_LOW 60000
#define HA_PORT 8123
#define HA_OTA_PATH "/local/ota/weather_gate/"

#define MQTT_TOPIC_WEATHER_DEFAULT "weather_gate/data"

// ===== LOG МАКРОС =====
#define LOG(tag, msg, ...) Serial.printf("[%08lu] [%s] " msg "\n", millis(), tag, ##__VA_ARGS__)

// ===== СТРУКТУРЫ (ДО ВСЕХ ФУНКЦИЙ!) =====
struct RollingHistory {
float temp[POINTS_HISTORY];
float pressure[POINTS_HISTORY];
float wind[POINTS_HISTORY];
int head = 0;
} history;

struct UpdateInfo {
String version;
String fw_url;
String fs_url;
String fw_md5;
String fs_md5;
String changelog;
};

// ===== ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ =====
bool rtc_found = false;
RTC_DS3231 rtc;

float temperature = 0, humidity = 0, pressure = 1013.25;
float wind_speed = 0, wind_gust = 0;
int wind_direction = 0;
float rain = 0;
unsigned long last_packet_time = 0;

float temp_min = 0, temp_max = 0;
float pressure_min = 1013.25, pressure_max = 1013.25;

```

```
float wind_max = 0;
```

```
float battery_voltage = 0;
```

```
float battery_coeff = 2.1;
```

```
float current_freq = 915.00;
```

```
bool verbose_mode = false;
```

```
String device_sn = "";
```

```
String hostname = "weather_gate";
```

```
String mqtt_ip = "", mqtt_user = "", mqtt_password = "";
```

```
String mqtt_topic_weather = MQTT_TOPIC_WEATHER_DEFAULT;
```

```
uint32_t radio_errors = 0, radio_success = 0;
```

```
unsigned long last_radio_time = 0;
```

```
uint32_t lastSaveTime = 0, lastHistoryTime = 0;
```

```
uint32_t lastMQTTReconnect = 0, lastMQTTPublish = 0;
```

```
// ===== ОБЪЕКТЫ =====
```

```
WebServer server(80);
```

```
Preferences pref;
```

```
WiFiClient ethClient;
```

```
PubSubClient mqtt(ethClient);
```

```
rtl_433_ESP rf;
```

```
char msgBuffer[512];
```

```
// ===== ПРОТОТИПЫ ФУНКЦИЙ =====
```

```
void loadSettings();
```

```
void saveAll();
```

```
void saveCounters();
```

```
void publishMQTTData();
```

```
void reconnectMQTT();
```

```
void readSensors();
```

```
void smartSave();
```

```
bool getUpdateInfo(UpdateInfo &info);
```

```
void performOTA();
```

```
bool handleFileRead(String path);
```

```
bool initCC1101();
```

```
String formatDateTime(DateTime dt);
```

```
float parseSafeFloat(String val);
```

```
String getBatteryStatus();
```

```
int getBatteryState();
```

```
void addHistory(float t, float p, float w);
```

```
void printResetReason();
```

```
// ===== ФУНКЦИИ =====

String formatDateTime(DateTime dt) {
    time_t utc = dt.unixtime();
    time_t local = utc + TIMEZONE_OFFSET * 3600;
    struct tm *timeinfo = localtime(&local);
    char buffer[20];
    strftime(buffer, sizeof(buffer), "%d.%m.%Y %H:%M:%S", timeinfo);
    return String(buffer);
}

float parseSafeFloat(String val) {
    val.replace(',', '.');
    val.trim();
    return val.toFloat();
}

String getBatteryStatus() {
    if (battery_voltage < 0.5) return "HET";
    if (battery_voltage < 1.0) return "HET";
    if (battery_voltage < 3.4 && !digitalRead(PIN_POWER_SENSE)) return "РАЗРЯЖЕН";
    return "OK";
}

int getBatteryState() {
    if (battery_voltage < 0.5) return 0;
    if (battery_voltage < 1.0) return 0;
    if (battery_voltage < 3.4 && !digitalRead(PIN_POWER_SENSE)) return 1;
    return 2;
}

void addHistory(float t, float p, float w) {
    if (!digitalRead(PIN_POWER_SENSE)) t = NAN;
    history.head = (history.head + 1) % POINTS_HISTORY;
    history.temp[history.head] = t;
    history.pressure[history.head] = p;
    history.wind[history.head] = w;
}

void printResetReason() {
    esp_reset_reason_t reason = esp_reset_reason();
    LOG("SYS", "=== RESET DIAGNOSTIC ===");
    switch(reason) {
        case ESP_RST_POWERON: LOG("SYS", "Reset: POWER_ON"); break;
        case ESP_RST_EXT: LOG("SYS", "Reset: EXTERNAL"); break;
    }
}

```

```

case ESP_RST_SW: LOG("SYS", "Reset: SOFTWARE"); break;
case ESP_RST_PANIC: LOG("SYS", "Reset: PANIC"); break;
case ESP_RST_INT_WDT: LOG("SYS", "Reset: INT WDT"); break;
case ESP_RST_TASK_WDT: LOG("SYS", "Reset: TASK WDT"); break;
case ESP_RST_BROWNOUT: LOG("SYS", "Reset: BROWNOUT"); break;
default: LOG("SYS", "Reset: UNKNOWN (%d)", reason); break;
}
LOG("SYS", "Free heap: %u bytes", ESP.getFreeHeap());
LOG("SYS", "=====");
}

```

```

// ===== MQTT =====

```

```

void mqttCallback(char* topic, byte* payload, unsigned int length) {
String message;
for (int i = 0; i < length; i++) message += (char)payload[i];
}

```

```

void sendDiscovery(const char* name, const char* unit, const char* dev_cla) {
if (!mqtt.connected()) return;
String topic = "homeassistant/sensor/" + hostname + "/" + name + "/config";
DynamicJsonDocument doc(1024);
doc["name"] = hostname + " " + name;
doc["stat_t"] = ("smart/" + hostname + "/state").c_str();
doc["val_tpl"] = ("{{ value_json." + String(name) + " }}").c_str();
doc["uniq_id"] = (hostname + "_" + name).c_str();
doc["unit_of_meas"] = unit;
if (dev_cla) doc["dev_cla"] = dev_cla;
JsonObject dev = doc.createNestedObject("device");
dev["ids"] = hostname;
dev["name"] = "Weather Gate";
dev["sw"] = CURRENT_VERSION;
String out;
serializeJson(doc, out);
mqtt.publish(topic.c_str(), out.c_str(), true);
}

```

```

void registerAllSensors() {
sendDiscovery("temperature", "°C", "temperature");
sendDiscovery("humidity", "%", "humidity");
sendDiscovery("pressure", "hPa", "pressure");
sendDiscovery("wind_speed", "m/s", "wind_speed");
}

```

```

void publishMQTTData() {

```

```

if (!mqtt.connected()) return;
DynamicJsonDocument doc(1024);
doc["temperature"] = temperature;
doc["humidity"] = humidity;
doc["pressure"] = pressure;
doc["wind_speed"] = wind_speed;
doc["wind_gust"] = wind_gust;
doc["wind_direction"] = wind_direction;
doc["rain"] = rain;
doc["battery"] = battery_voltage;
doc["battery_status"] = getBatteryStatus();
doc["power_source"] = digitalRead(PIN_POWER_SENSE) ? "BAT" : "AC";
doc["uptime"] = millis() / 1000;
doc["radio_errors"] = radio_errors;
doc["radio_success"] = radio_success;
doc["current_freq"] = current_freq;
if (rtc_found) {
doc["datetime"] = formatDateTime(rtc.now());
}
String out;
serializeJson(doc, out);
mqtt.publish(("smart/" + hostname + "/state").c_str(), out.c_str(), false);
LOG("MQTT", "State published (%d bytes)", out.length());
}

void reconnectMQTT() {
if (mqtt_ip.length() < 7) return;
String clientId = "WeatherGate_" + hostname + "_" + String(ESP.getEfuseMac(), HEX);
if (mqtt.connect(clientId.c_str(), mqtt_user.c_str(), mqtt_password.c_str())) {
LOG("MQTT", "Connected");
registerAllSensors();
} else {
LOG("MQTT", "Failed, rc=%d", mqtt.state());
}
}

// ===== RTL_433 CALLBACK =====
void rtl_433_Callback(char* message) {
digitalWrite(LED_RF_RX, HIGH);
JsonDocument doc;
DeserializationError err = deserializeJson(doc, message);
if (!err) {
radio_success++;
last_radio_time = millis();
}
}

```

```

last_packet_time = millis();
if (doc.containsKey("temperature_C")) temperature = doc["temperature_C"];
else if (doc.containsKey("temp_C")) temperature = doc["temp_C"];
if (doc.containsKey("humidity")) humidity = doc["humidity"];
if (doc.containsKey("pressure_hPa")) pressure = doc["pressure_hPa"];
if (doc.containsKey("wind_avg_m_s")) wind_speed = doc["wind_avg_m_s"];
if (doc.containsKey("wind_max_m_s")) wind_gust = doc["wind_max_m_s"];
if (doc.containsKey("wind_dir_deg")) wind_direction = doc["wind_dir_deg"];
if (doc.containsKey("rain_mm")) rain = doc["rain_mm"];
if (temperature < temp_min || temp_min == 0) temp_min = temperature;
if (temperature > temp_max) temp_max = temperature;
if (pressure < pressure_min) pressure_min = pressure;
if (pressure > pressure_max) pressure_max = pressure;
if (wind_speed > wind_max) wind_max = wind_speed;
LOG("WS1080", "T=%.1f°C H=%.0f%% P=%.0fhPa W=%.1fm/s Dir=%d°",
temperature, humidity, pressure, wind_speed, wind_direction);
if (mqtt.connected()) {
digitalWrite(LED_MQTT_TX, HIGH);
mqtt.publish(("smart/" + hostname + "/state").c_str(), message);
delay(20);
digitalWrite(LED_MQTT_TX, LOW);
}
} else {
radio_errors++;
LOG("RTL", "Parse error: %s", err.c_str());
}
digitalWrite(LED_RF_RX, LOW);
}

// ===== ЧТЕНИЕ ДАТЧИКОВ =====
void readSensors() {
int raw = 0;
for (int i = 0; i < 5; i++) raw += analogRead(PIN_BAT_ADC);
battery_voltage = (raw / 5.0) * (3.3 / 4095.0) * battery_coeff;
if (millis() - last_packet_time > 3600000) {
LOG("WS1080", "No data for 1 hour, checking connection...");
}
}

// ===== УМНОЕ СОХРАНЕНИЕ =====
void smartSave() {
int state = getBatteryState();
unsigned long saveInterval = SMART_SAVE_NORMAL;
if (state == 1) saveInterval = SMART_SAVE_LOW;

```

```

if (millis() - lastSaveTime >= saveInterval) {
    saveCounters();
    lastSaveTime = millis();
    LOG("SYS", "Smart save");
}
}

// ===== НАСТРОЙКИ =====
void loadSettings() {
    pref.begin("weather", true);
    device_sn = pref.getString("sn", "");
    hostname = pref.getString("hostname", "weather_gate");
    mqtt_ip = pref.getString("mq_ip", "");
    mqtt_user = pref.getString("mq_u", "");
    mqtt_password = pref.getString("mq_p", "");
    current_freq = pref.getFloat("freq", 915.0);
    battery_coeff = pref.getFloat("bat_k", 2.1);
    radio_errors = pref.getUInt("radio_errors", 0);
    radio_success = pref.getUInt("radio_success", 0);
    temperature = pref.getFloat("temp", 0);
    humidity = pref.getFloat("hum", 0);
    pressure = pref.getFloat("pres", 1013.25);
    temp_min = pref.getFloat("temp_min", 0);
    temp_max = pref.getFloat("temp_max", 0);
    pressure_min = pref.getFloat("pres_min", 1013.25);
    pressure_max = pref.getFloat("pres_max", 1013.25);
    wind_max = pref.getFloat("wind_max", 0);
    pref.end();
    LOG("SYS", "Settings loaded. Hostname: %s", hostname.c_str());
    LOG("SYS", "Current frequency: %.2f MHz", current_freq);
}

void saveAll() {
    pref.begin("weather", false);
    pref.putString("sn", device_sn);
    pref.putString("hostname", hostname);
    pref.putString("mq_ip", mqtt_ip);
    pref.putString("mq_u", mqtt_user);
    pref.putString("mq_p", mqtt_password);
    pref.putFloat("freq", current_freq);
    pref.putFloat("bat_k", battery_coeff);
    pref.putUInt("radio_errors", radio_errors);
    pref.putUInt("radio_success", radio_success);
    pref.putFloat("temp", temperature);
}

```



```

pref.putFloat("hum", humidity);
pref.putFloat("pres", pressure);
pref.putFloat("temp_min", temp_min);
pref.putFloat("temp_max", temp_max);
pref.putFloat("pres_min", pressure_min);
pref.putFloat("pres_max", pressure_max);
pref.putFloat("wind_max", wind_max);
pref.end();
LOG("SYS", "All settings saved");
}

```

```

void saveCounters() {
pref.begin("weather", false);
pref.putUInt("radio_errors", radio_errors);
pref.putUInt("radio_success", radio_success);
pref.putFloat("temp", temperature);
pref.putFloat("hum", humidity);
pref.putFloat("pres", pressure);
pref.putFloat("temp_min", temp_min);
pref.putFloat("temp_max", temp_max);
pref.putFloat("pres_min", pressure_min);
pref.putFloat("pres_max", pressure_max);
pref.putFloat("wind_max", wind_max);
pref.end();
}

```

```

// ===== OTA =====
bool getUpdateInfo(UpdateInfo &info) {
if (mqtt_ip.length() < 7) return false;
String url = "http://" + mqtt_ip + ":" + String(HA_PORT) + HA_OTA_PATH + "version.json";
HTTPClient http;
http.setTimeout(5000);
if (!http.begin(url)) return false;
int httpCode = http.GET();
if (httpCode != HTTP_CODE_OK) {
http.end();
return false;
}
String payload = http.getString();
http.end();
StaticJsonDocument<768> doc;
DeserializationError error = deserializeJson(doc, payload);
if (error) return false;
info.version = doc["version"] | "";
}

```

```

info.fw_md5 = doc["fw_md5"] | "";
info.fs_md5 = doc["fs_md5"] | "0";
info.changelog = doc["changelog"] | "";
String baseUrl = "http://" + mqtt_ip + ":" + String(HA_PORT) + HA_OTA_PATH;
info.fw_url = baseUrl + "firmware.bin";
info.fs_url = baseUrl + "littelfs.bin";
LOG("OTA", "Version: %s", info.version.c_str());
return info.version.length() > 0;
}

```

```

void performOTA() {
LOG("OTA", "=== OTA START ===");
UpdateInfo info;
if (!getUpdateInfo(info)) return;
if (info.version == CURRENT_VERSION) return;
LOG("OTA", "Updating to %s", info.version.c_str());
mqtt.disconnect();
delay(100);
WiFiClient otaClient;
otaClient.setTimeout(12000);
if (info.fs_md5.length() > 0 && info.fs_md5 != "0") {
LOG("OTA", "Updating LittleFS...");
httpUpdate.setMD5sum(info.fs_md5);
httpUpdate.updateFs(otaClient, info.fs_url);
delay(500);
}
LOG("OTA", "Updating firmware...");
httpUpdate.setMD5sum(info.fw_md5);
httpUpdate.rebootOnUpdate(true);
httpUpdate.update(otaClient, info.fw_url);
}

```

```

// ===== GZIP ОБРАБОТКА =====
bool handleFileRead(String path) {
if (path.endsWith("/")) path += "index.html";
String contentType = "text/html";
if (path.endsWith(".css")) contentType = "text/css";
else if (path.endsWith(".js")) contentType = "application/javascript";
else if (path.endsWith(".json")) contentType = "application/json";
else if (path.endsWith(".ico")) contentType = "image/x-icon";
String gzPath = path + ".gz";
if (LittleFS.exists(gzPath)) {
File file = LittleFS.open(gzPath, "r");
server.setContentLength(file.size());
}
}

```

```

server.setHeader("Content-Encoding", "gzip");
server.send(200, contentType, "");
uint8_t buffer[512];
size_t bytesRead;
while ((bytesRead = file.read(buffer, sizeof(buffer))) > 0) {
server.client().write(buffer, bytesRead);
}
file.close();
return true;
}
if (LittleFS.exists(path)) {
File file = LittleFS.open(path, "r");
server.streamFile(file, contentType);
file.close();
return true;
}
return false;
}

// ===== ИНИЦИАЛИЗАЦИЯ CC1101 =====
bool initCC1101() {
LOG("CC1101", "Initializing with FSK parameters for WS1080...");
ELECHOUSE_cc1101.setSpiPin(CC1101_SCK, CC1101_MISO, CC1101_MOSI, CC1101_CS);
if (!ELECHOUSE_cc1101.getCC1101()) {
LOG("CC1101", "Chip not detected!");
return false;
}
LOG("CC1101", "Chip detected");
ELECHOUSE_cc1101.Init();
ELECHOUSE_cc1101.setMHZ(current_freq); // setMHZ (заглавные HZ!)
ELECHOUSE_cc1101.setModulation(0); // 0 = 2-FSK
ELECHOUSE_cc1101.setDRate(17.24); // 17.24 kBaud
ELECHOUSE_cc1101.setDeviation(47.6); // 47.6 kHz
ELECHOUSE_cc1101.setRxBW(325.0); // 325 kHz
ELECHOUSE_cc1101.SpiWriteReg(CC1101_IOCFG0, 0x0D); // GDO0 = RAW data
ELECHOUSE_cc1101.SpiWriteReg(CC1101_PKTCTRL0, 0x32); // Async mode
LOG("CC1101", "Configured: %.2f MHz, 2-FSK, DRate=17.24k, Dev=47.6kHz, BW=325kHz", current_freq);
return true;
}

// ===== SETUP =====
void setup() {
Serial.begin(115200);
delay(500);

```

```
printResetReason();
pinMode(LED_RF_RX, OUTPUT);
pinMode(LED_MQTT_TX, OUTPUT);
pinMode(PIN_POWER_SENSE, INPUT);
digitalWrite(LED_RF_RX, LOW);
digitalWrite(LED_MQTT_TX, LOW);
analogSetAttenuation(ADC_11db);
// RTC
Wire.begin(I2C_SDA, I2C_SCL);
if (rtc.begin()) {
  rtc_found = true;
  LOG("RTC", "DS3231 found");
} else {
  LOG("RTC", "DS3231 NOT found");
}
// LittleFS
if (!LittleFS.begin(true)) {
  LOG("FS", "LittleFS mount failed");
} else {
  LOG("FS", "LittleFS mounted");
}
// Ethernet
WiFi.mode(WIFI_OFF);
btStop();
ETH.begin(ETH_PHY_LAN8720, 1, 23, 18, 16, ETH_CLOCK_GPIO17_OUT);
if (hostname.length() > 0) ETH.setHostname(hostname.c_str());
delay(500);
unsigned long startWait = millis();
while (ETH.localIP().toString() == "0.0.0.0" && millis() - startWait < 10000) {
  delay(100);
}
LOG("ETH", "IP address: %s", ETH.localIP().toString().c_str());
loadSettings();
// CC1101
if (!initCC1101()) {
  LOG("CC1101", "WARNING: CC1101 initialization failed!");
}
// rtl_433
rf.setCallback(rtl_433_Callback, msgBuffer, 512);
rf.rtlVerbose = verbose_mode ? 1 : 0;
rf.enableReceiver();
LOG("RF", "rtl_433 receiver enabled");
// MQTT
if (mqtt_ip.length() > 5) {
```

```

mqtt.setServer(mqtt_ip.c_str(), 1883);
mqtt.setCallback(mqttCallback);
LOG("MQTT", "Server set to %s", mqtt_ip.c_str());
}
// ===== WEB-CEPBEP =====
server.on("/api/data", HTTP_GET, []() {
DynamicJsonDocument doc(2048);
doc["temperature"] = temperature;
doc["humidity"] = humidity;
doc["pressure"] = pressure;
doc["wind_speed"] = wind_speed;
doc["wind_gust"] = wind_gust;
doc["wind_direction"] = wind_direction;
doc["rain"] = rain;
doc["temp_min"] = temp_min;
doc["temp_max"] = temp_max;
doc["pressure_min"] = pressure_min;
doc["pressure_max"] = pressure_max;
doc["wind_max"] = wind_max;
doc["battery"] = battery_voltage;
doc["battery_status"] = getBatteryStatus();
doc["power_source"] = digitalRead(PIN_POWER_SENSE) ? "BAT" : "AC";
doc["mqtt_status"] = mqtt.connected() ? "OK" : "Offline";
doc["hostname"] = hostname;
doc["version"] = CURRENT_VERSION;
doc["uptime"] = millis() / 1000;
doc["radio_errors"] = radio_errors;
doc["radio_success"] = radio_success;
doc["current_freq"] = current_freq;
doc["verbose_mode"] = verbose_mode;
if (rtc_found) doc["datetime"] = formatDateTime(rtc.now());
String buf;
serializeJson(doc, buf);
server.send(200, "application/json", buf);
});
server.on("/api/history", HTTP_GET, []() {
DynamicJsonDocument doc(4096);
JSONArray tempHist = doc.createNestedArray("temp_history");
JSONArray pressureHist = doc.createNestedArray("pressure_history");
JSONArray windHist = doc.createNestedArray("wind_history");
for (int i = 1; i <= POINTS_HISTORY; i++) {
int idx = (history.head + i) % POINTS_HISTORY;
tempHist.add(isnan(history.temp[idx]) ? (float)NULL : history.temp[idx]);
pressureHist.add(isnan(history.pressure[idx]) ? (float)NULL : history.pressure[idx]);
}
}

```

```

windHist.add(isnan(history.wind[idx]) ? (float)NULL : history.wind[idx]);
}
String buf;
serializeJson(doc, buf);
server.send(200, "application/json", buf);
});
server.on("/api/sync_time", HTTP_POST, []() {
if (server.hasArg("t") && rtc_found) {
rtc.adjust(DateTime(server.arg("t").toInt()));
server.send(200, "text/plain", "OK");
} else {
server.send(400, "text/plain", "Failed");
}
});
server.on("/api/set_freq", HTTP_POST, []() {
if (server.hasArg("f")) {
float new_freq = server.arg("f").toFloat();
if (new_freq >= 860.0 && new_freq <= 930.0) {
current_freq = new_freq;
pref.begin("weather", false);
pref.putFloat("freq", current_freq);
pref.end();
ELECHOUSE_cc1101.setMHZ(current_freq);
LOG("RF", "Frequency changed to %.2f MHz", current_freq);
server.send(200, "text/plain", "OK");
} else {
server.send(400, "text/plain", "Invalid frequency");
}
} else {
server.send(400, "text/plain", "Missing frequency");
}
});
server.on("/api/set_verbose", HTTP_POST, []() {
if (server.hasArg("v")) {
verbose_mode = (server.arg("v") == "1");
rf.rtlVerbose = verbose_mode ? 1 : 0;
server.send(200, "text/plain", "OK");
}
});
server.on("/api/reset_radio_errors", HTTP_POST, []() {
radio_errors = 0;
radio_success = 0;
saveCounters();
server.send(200, "text/plain", "OK");
}

```

```

});
server.on("/api/config", HTTP_POST, []() {
String body = server.arg("plain");
DynamicJsonDocument doc(2048);
if (!deserializeJson(doc, body)) {
if (doc.containsKey("hostname")) hostname = doc["hostname"].as<String>();
if (doc.containsKey("mq_ip")) mqtt_ip = doc["mq_ip"].as<String>();
if (doc.containsKey("mq_u")) mqtt_user = doc["mq_u"].as<String>();
if (doc.containsKey("mq_p")) mqtt_password = doc["mq_p"].as<String>();
if (doc.containsKey("bat_k")) battery_coeff = parseSafeFloat(doc["bat_k"].as<String>());
ETH.setHostname(hostname.c_str());
mqtt.setServer(mqtt_ip.c_str(), 1883);
saveAll();
server.send(200, "OK");
delay(500);
ESP.restart();
} else {
server.send(400, "text/plain", "JSON Error");
}
});

server.onNotFound([]() {
if (!handleFileRead(server.uri())) {
server.send(404, "text/plain", "Not Found");
}
});

server.begin();
LOG("HTTP", "Server started");
for (int i = 0; i < POINTS_HISTORY; i++) {
history.temp[i] = 0;
history.pressure[i] = 0;
history.wind[i] = 0;
}
lastHistoryTime = millis();
lastSaveTime = millis();
LOG("SYS", "Setup complete");
LOG("SYS", "Waiting for WS1080 packets at %.2f MHz (FSK, 17.24kbps)...", current_freq);
}

// ===== LOOP =====
void loop() {
server.handleClient();
yield();
static uint32_t lastSensors = 0;
static uint32_t lastRfTask = 0;

```

```
if (millis() - lastSensors >= SENSOR_INTERVAL) {
  lastSensors = millis();
  readSensors();
}
if (millis() - lastRfTask >= 10) {
  lastRfTask = millis();
  rf.loop();
}
smartSave();
if (millis() - lastHistoryTime >= HISTORY_INTERVAL) {
  lastHistoryTime = millis();
  addHistory(temperature, pressure, wind_speed);
}
if (!mqtt.connected() && mqtt_ip.length() > 5 && millis() - lastMQTTReconnect >=
MQTT_RECONNECT_INTERVAL) {
  lastMQTTReconnect = millis();
  reconnectMQTT();
}
if (mqtt.connected() && millis() - lastMQTTPublish >= MQTT_PUBLISH_INTERVAL) {
  lastMQTTPublish = millis();
  publishMQTTData();
}
mqtt.loop();
delay(1);
}
```